



Literature Results

Objective

I am developing a real-time closed-loop system for detecting interictal epileptiform discharges (IEDs) and triggering stimulation on a microcontroller. The target board is an STM32 NUCLEO-H723ZG, so the detection method must be computationally lightweight enough to run in real time on an embedded Cortex-M7 MCU with limited RAM, limited CPU headroom, and no GPU.

Please conduct a focused literature and engineering review on the best methods for IED detection that are suitable for closed-loop control on a microcontroller.

Main objective:

Identify IED detection algorithms that can achieve high sensitivity and high specificity while being simple enough to implement in embedded C/C++ on the STM32 NUCLEO-H723ZG.

Prioritize methods that are:

1. Real-time and causal, not offline-only.
2. Computationally lightweight.
3. Robust to baseline drift, noise, stimulation artifact, and amplitude variation.
4. Suitable for hippocampal/thalamic/local field potential or intracranial electrophysiology recordings.
5. Easy to tune and validate experimentally.
6. Appropriate for closed-loop stimulation, where detection latency matters.

Please compare methods such as:

- * Amplitude thresholding after bandpass filtering
- * Line length
- * RMS / short-time energy
- * Teager-Kaiser energy operator / nonlinear energy operator
- * Envelope detection using rectification + moving average
- * Bandpower ratios
- * Derivative/slope-based detection
- * Template matching
- * Wavelet-based methods, only if lightweight enough
- * Simple statistical adaptive thresholds, such as median/MAD or running mean/standard deviation
- * Hybrid detectors combining multiple simple features
- * Lightweight ML methods only if they are genuinely feasible on an STM32, such as logistic regression, shallow decision trees, or small fixed-point classifiers

For each method, summarize:

- * How the algorithm works
- * Whether it is causal and real-time
- * Expected sensitivity/specificity from the literature, when available
- * Detection latency
- * Computational cost

- * Memory requirements
- * Parameter tuning strategy
- * Strengths and weaknesses
- * Susceptibility to noise/artifact
- * Whether it is appropriate for closed-loop stimulation
- * Whether it can be implemented efficiently in embedded C on an STM32H723ZG

Please rank the methods from most to least promising for my use case.

Also provide:

1. A recommended final detection pipeline for STM32 implementation.
2. Pseudocode for the recommended detector.
3. Suggested filter bands for IED detection.
4. Suggested feature window sizes in milliseconds.
5. Suggested adaptive thresholding approach.
6. Artifact rejection strategy.
7. Refractory/lockout period recommendations.
8. How to validate sensitivity, specificity, false positives per minute, and detection latency.
9. How to tune the detector using manually labeled IEDs.
10. A practical embedded implementation plan, including fixed-point vs floating-point considerations.

My current goal is not necessarily to use the most advanced algorithm. I want the best tradeoff between detection accuracy, low false positives, low latency, and reliable real-time performance on the STM32 NUCLEO-H723ZG.

Please cite primary papers, review papers, and any examples of embedded or closed-loop seizure/IED detection systems. Distinguish clearly between methods validated for seizures and methods validated specifically for IEDs. Also distinguish methods that only work offline from methods that are suitable for real-time closed-loop control.

Results

Focused literature + engineering review: microcontroller-suitable IED detection

Executive engineering takeaways

A practical embedded IED detector on an STM32H723ZG (Cortex-M7, no GPU) should (i) be causal, (ii) use O1 per-sample updates with short windows ($\approx 8\text{--}32$ ms) for spike-like transients, (iii) use adaptive thresholds updated on background-only epochs to handle baseline drift and amplitude variability, and (iv) include aggressive stimulation-artifact blanking + refractory lockout.

The strongest “embedded-credible” evidence for lightweight real-time detectors comes from the NeuroPace RNS closed-loop implant, whose detection toolkit is explicitly constrained by implantable power/compute and uses three simple detectors—**line length**, **area (energy-like)**, and **bandpass half-wave counting**—run in real time at 250 Hz ([1.1], [2.1]). These tools are directly relevant to your goal because they were designed for real-time closed-loop stimulation under tight compute constraints and provide design patterns you can port to an STM32.

Evidence-based constraints (what matters for MCU closed-loop)

- **Causality and latency:** Use short windows (order 10–30 ms) for IEDs; latency is dominated by the feature window + filter delay. RNS detectors are explicitly designed to detect events from fractions of a second up to several seconds via short-vs-long windows, but the underlying computations are causal sliding-window comparisons ([2.1]).
- **Compute:** Favor features implementable with adds, abs, and a few multiplies. Hardware/algorithm co-design work emphasizes that FFT/DWT and memory-heavy operations raise power/memory cost, while time-domain features

(energy, RMS, coastline/line length) are low complexity and feasible for implantables and standard microcontrollers ([3.1], [3.2]).

- **Robustness:** For closed-loop stimulation, explicitly handle artifacts. The RNS literature highlights that detection thresholds are set relative to longer-term background and that detectors can be combined with persistence rules—this is one way to improve robustness without heavy computation ([1.2], [2.1]).

Ranked methods for your use case (most → least promising)

The ranking below is driven by: causal real-time feasibility, robustness under stimulation artifacts, and evidence of use in implantable/closed-loop systems.

Method	Causal?	Typical latency driver	Compute cost	Memory	Robustness to drift / noise / artifact	Ease of tuning	Evidence type	Overall score
Line length	Yes	Short window, typically ~8–40 ms for spikes; RNS-style short-vs-long windows can be 128 ms–4 s vs 4 s–16 min depending on target event ([2.1], [1.3])	Very low; $O(N)$, one abs-diff + accumulate/sample; among lowest on-chip power/latency ([4.1])	Tiny; ring buffer + running sum	Good against slow drift; sensitive to broadband artifact/EMG/stim artifact unless gated; captures amplitude + frequency changes well ([1.2], [1.3])	Very easy; threshold + window + refractory	Strong seizure/implant evidence in intracranial closed-loop systems; indirect but compelling for IED-like transients via RNS and hardware reviews ([1.2], [2.1], [3.2])	1
Bandpass half-wave (RNS-style)	Yes	Half-wave duration and count window; can be very low-latency once a few qualifying half-waves occur ([1.1], [2.1])	Low; $O(N)$, local extrema tracking + amplitude/duration tests	Tiny to small; extrema state + counters	Better specificity than raw amplitude; robust to drift after bandpass; still vulnerable to sharp artifacts/stim pulses unless blanked ([1.1], [2.1])	Easy-moderate; amplitude, duration, count thresholds	Strong intracranial closed-loop seizure evidence from RNS; conceptually close to IED morphology detection ([1.1], [2.1])	2
Amplitude threshold after bandpass	Yes	Filter group delay + confirmation window, often ~5–30 ms	Very low; $O(N)$, filter + compare	Tiny	Good against drift if HP/bandpass used; poor alone for amplitude variation and sharp artifact discrimination	Very easy	Classic spike/IED logic in intracranial studies with slope/duration/amplitude rules; also common as baseline comparator ([5.1], [6.1], [5.2])	3
RMS / short-time energy / area	Yes	Window length, usually ~8–32 ms for spikes or longer for seizures; area detector in RNS uses windowed energy-like measure ([1.2], [1.3])	Very low; $O(N)$, square/abs + running sum	Tiny	Good against drift after bandpass; less shape-specific than line length; broad artifacts can drive false positives	Very easy	Strong seizure/implant evidence; energy/area explicitly implemented in RNS and hardware co-design literature ([1.2], [3.1], [1.3])	4
Envelope (rectify + moving average)	Yes	Smoothing window, often ~5–20 ms beyond filter delay	Very low; $O(N)$, abs + MA	Tiny	Good for amplitude variation and smoother thresholds; can smear onset and be artifact-sensitive	Very easy	More common in seizure-feature sets than dedicated IED validation; envelope used as simple feature in seizure work ([7.1])	5
TKEO / NEO / SNEO	Quasi-causal; classic TKEO uses $x[n+1]$, so 1-sample look-ahead unless delayed by 1 sample	Essentially sample-level plus any smoothing/threshold window; very low intrinsic latency	Very low; $O(N)$, few multiplies/adds per sample	Tiny	Enhances sharp transients; can be noise-sensitive without smoothing/robust thresholding; strong response to stimulation artifact	Easy-moderate; smoothing and adaptive threshold matter	Quantified mainly in seizure detection and periodic discharge detection; threshold TKEO gave 66.9–91.3% sensitivity and 60.8–97.5% specificity in mouse seizure models; TKEO+DWT 73.2–80.1% / 75.8–98.1% ([7.2]). Used to localize peaks of epileptiform discharges in EEG ([7.1])	6

Method	Causal?	Typical latency driver	Compute cost	Memory	Robustness to drift / noise / artifact	Ease of tuning	Evidence type	Overall score
Slope / derivative	Yes	1–5 samples plus optional short confirmation window	Very low; $O(N)$, subtraction/abs	Tiny	Sensitive to sharp spikes but also to high-frequency noise and stim artifact; needs bandpass and refractory logic	Easy	Longstanding IED/spike logic; inclining/declining slope and duration used in intracranial spike detectors/clustering workflows ([5.1], [6.1], [5.2])	7
Lightweight ML (logistic regression, small tree)	Yes	Feature window length dominates; classifier latency negligible	Low to moderate; $O(F)$ for logistic regression, $O(\text{tree depth})$ for small tree	Small; feature vector + weights/tree	Can improve specificity by combining cheap features; robustness depends on training drift and patient specificity	Moderate; requires labeled data and retraining policy	Strong seizure/embedded evidence; reviews note tree/threshold features fit on-chip constraints, and low-power online logistic regression is feasible ([8.1], [3.2]). Limited direct IED-on-MCU evidence	8
Bandpower ratios (Goertzel/FFT)	Yes	Window length usually 64–512 ms+ for stable spectra	Moderate; $O(NK)$ with Goertzel or $O(N \log N)$ FFT	Small-moderate; buffers + spectral bins	More robust to amplitude drift; weaker for very brief spikes unless windows are short, which harms frequency resolution; line noise handling needed	Moderate	Mostly seizure/predictor evidence; bandpower features listed on-chip with 0.25–1 s latencies and much higher power than line length ([4.1], [8.2])	9
AR prediction error (Dümpelmann two-stage)	Yes	Predictor update interval + averaging window for prediction error; coefficients updated every 4 s in study ([5.1])	Moderate; $O(p)$ per sample with $p \approx 10$ plus running error stats	Small-moderate; AR coeffs + state + error windows	Good for detecting deviation from local background; may handle drift better than fixed thresholds, but not ideal under abrupt artifact/stim changes	Moderate-hard	Direct intracranial IED evidence; two-stage linear predictor had best reviewer agreement among compared automatic methods in one study (32% vs 24–26% for alternatives) ([5.2]); model order 10 used to reduce computation ([5.1])	10
Template matching (DTW)	Usually no for strict MCU closed-loop if full DTW; simple matched filter can be causal, DTW is too heavy	Template length + search horizon	High for DTW; worse than $O(N)$ simple features	Moderate-high; templates + warping state	Potentially high specificity to patient morphology; poor robustness to shape drift unless templates updated; artifact confusion possible	Moderate-hard	Strong for semi-automated IED annotation, not embedded closed-loop; DTW annotation reduced expert time ~70% but was not positioned as implantable real-time detection ([9.1], [9.2], [9.3])	11
Wavelet (DWT/CWT)	Yes in principle	Decomposition level and frame size; often tens of ms to >100 ms	Moderate (DWT) to high (CWT); DWT $O(N)$, CWT high constant cost	Small-moderate (DWT) to high (CWT)	Good time-frequency localization and denoising; can improve specificity; artifact robustness depends on bands/features	Moderate-hard	Real-time spike DSP implementation exists (wavelet + dedicated DSP card) but literature often omits memory/latency details; more practical for DSP/SoC than bare MCU if kept very small ([10.1], [10.2], [11.1])	12

Table: This table ranks practical IED/spike detection methods for real-time closed-loop deployment on a Cortex-M7, emphasizing causal operation, low compute and memory cost, and evidence from intracranial or implantable systems. It is useful for narrowing candidates to methods that best balance sensitivity, specificity, latency, and STM32 implementation feasibility.

Key literature anchor: RNS implements **line length**, **area**, and **bandpass half-wave** tools because they are computationally efficient in an implantable device and can be combined for higher specificity ([1.1], [2.1]).

Method-by-method technical summary (focused on MCU feasibility)

1) Line length (LL)

How it works. Compute average absolute sample-to-sample difference over a short window: ($LL = \frac{1}{N} \sum |x[n] - x[n-1]|$). In RNS, a short-term LL is compared to a long-term background LL; detection triggers when short-term exceeds an absolute or relative threshold, and negative thresholds can detect decreases (electrodecrement) ([1.2], [1.1]).

Real-time/causal. Yes; O1 per-sample update via a running sum.

Latency. $\approx$ filter group delay + LL window (typically 8–32 ms for IED use). RNS uses short windows as low as 128 ms for its general detector framework ([2.1]), but for IED triggering you can reduce short windows substantially.

Compute/memory. Extremely low; a ring buffer of recent |diff| to maintain a running sum.

Tuning. Choose LL window (ms) and threshold multiplier (e.g., median+K·MAD); optionally require persistence for 1–2 hops.

Strengths. Robust to slow drift; captures amplitude and frequency change simultaneously ([1.2]).

Weaknesses. Sensitive to any sharp artifact (including stimulation), so blanking/gating is mandatory in closed-loop.

Closed-loop suitability. Excellent; directly motivated by implantable closed-loop detectors ([1.2], [2.1]) and low-power feature tables ([4.1]).

2) Bandpass half-wave counting (RNS-style “bandpass tool”)

How it works. Segment the bandpassed waveform into **half-waves** by tracking local minima/maxima; count half-waves that exceed programmable **amplitude** and **duration** limits; trigger if the count in a window exceeds a threshold ([1.1], [1.3]).

Real-time/causal. Yes; based on local extrema.

Latency. Often low, because detection can occur after only a small number of qualifying half-waves (1–3), depending on thresholds.

Compute/memory. Low; derivative sign-change tracking plus counters.

Tuning. Half-wave duration bounds (ms), amplitude threshold, count threshold, and window.

Strengths. More morphology/oscillation specific than pure amplitude/energy; RNS uses it to provide more specific detection once target bands are known ([1.2]).

Weaknesses. Still highly sensitive to stimulation pulses unless blanked.

Closed-loop suitability. Excellent; explicitly used in closed-loop implant ([1.1], [2.1]).

3) Bandpass + amplitude threshold (with slope/duration gates)

How it works. After bandpass, detect peaks crossing an amplitude threshold; optionally gate with slope and duration constraints.

Evidence. Intracranial spike detectors include amplitude threshold sweeps and criteria such as sharpness and durations of inclining/declining components ([5.1]). Hufnagel et al. describe spike quantification using minima within ± 100 ms and slope-derived amplitude measures in a 200 ms frame ([6.1]).

Real-time/causal. Yes.

Compute/memory. Very low.

Strengths. Simple, transparent.

Weaknesses. Amplitude variation across time/contacts can destabilize thresholds; needs robust adaptive thresholding.

Closed-loop suitability. Good as a component, but typically inferior to LL/half-wave hybrids for specificity.

4) Short-time energy / RMS / “Area” detector

How it works. Windowed energy-like measures using squaring or absolute area-under-curve. RNS includes an “area” detector (average area under the curve within a window) and uses short-vs-long window comparisons like LL ([1.2], [1.3]).

Real-time/causal. Yes.

Compute/memory. Very low.

Strengths. Drift-resistant after bandpass; simplest to implement.

Weaknesses. Less selective for spike morphology than half-wave/LL; can false-trigger on broadband artifacts.

Closed-loop suitability. Good, especially paired with LL or half-wave.

5) Envelope detection (rectification + moving average)

How it works. Compute $|\text{bandpassed } x|$ and smooth with a moving average or EWMA.

Evidence. Envelope is included among simple features used in seizure work (Hilbert-based envelope) ([7.1]).

Real-time/causal. Yes.

Strengths. Smooths noisy amplitude; easier thresholding.

Weaknesses. Adds smoothing latency; not very shape-specific.

6) Teager–Kaiser Energy Operator (TKEO) / NEO family

How it works. Classic discrete TKEO: $\Psi[x(n)] = x(n)^2 - x(n-1)x(n+1)$ ([7.1]). This highlights sharp, oscillatory transients.

Real-time/causal. Nearly; the standard form uses $x[n+1]$, but you can implement with a 1-sample delay (effectively causal with minimal latency).

Performance evidence (mostly seizures). In mouse seizure detection, a TKEO threshold method showed sensitivity 66.9–91.3% and specificity 60.8–97.5% depending on model; adding DWT modestly improved performance ([7.2]). This is seizure evidence, not IED-specific.

Strengths. Very low compute; strong response to transient changes.

Weaknesses. Often too sensitive to high-frequency noise and especially to stimulation artifact; requires smoothing and robust thresholds.

7) AR prediction-error (Dümpelmann two-stage)

How it works. Fit a low-order AR predictor to background EEG and detect spikes as increases in prediction error; in one intracranial evaluation, AR coefficients were updated every 4 s and prediction error was squared/averaged and compared against surrounding windows ([5.1]).

Evidence (intracranial spikes). In Dümpelmann & Elger's intracranial spike comparison, agreement with human reviewers was low for all methods, but the two-stage predictor had higher agreement (32%) than Gotman (24%) or a wavelet detector (26%); interrater agreement between humans was only 40%, emphasizing intrinsic ambiguity ([5.2]).

Suitability. Feasible on MCU ($p \approx 10$) but more complex than LL/half-wave and still vulnerable to artifacts.

8) Bandpower / bandpower ratios (Goertzel/FFT)

How it works. Compute power in bands and ratios.

Embedded cost evidence. On-chip comparisons indicate bandpower features have materially higher power and longer latencies than LL (0.25–1 s listed latencies vs LL 0.25 s in that table) ([4.1]). For IEDs (brief), spectral windows short enough to reduce latency also reduce frequency resolution.

Suitability. Better for seizures than for brief IEDs; not first choice for low-latency IED-triggered stimulation.

9) Template matching / DTW

How it works. Match candidate waveforms to templates; DTW allows time warping.

Evidence. DTW template matching has been used to accelerate expert IED annotation (average ~70% time reduction) and build large IED databases, often with downsampling (e.g., 128 Hz) to reduce compute ([9.3]). This is valuable for offline labeling/tuning, not MCU closed-loop.

Suitability. DTW is too heavy for deterministic, low-latency MCU unless heavily constrained.

10) Wavelet methods

How it works. CWT/DWT features or wavelet energy.

Real-time evidence. Real-time wavelet spike detection has been implemented on a dedicated floating-point DSP board (Motorola 96002) using the Morlet wavelet, showing feasibility with special hardware ([10.1], [10.2]). Reviews note wavelets are effective but computational feasibility and memory/latency are often not reported ([11.1]).

Suitability. A tiny DWT might be feasible on STM32, but you generally do not need it if LL+half-wave achieves acceptable specificity.

11) Lightweight ML (logistic regression / small trees)

How it works. Combine a few cheap features (LL, energy, half-wave counts, etc.) with a linear classifier or small tree.

Embedded evidence. Reviews of closed-loop/on-chip intelligence emphasize that interpretable tree-based models and lightweight features are practical for closed-loop stimulation ([4.1]). Survey work notes that patient-specific small feature sets can be competitive with more complex methods ([8.1]). However, high-quality IED-labeled intracranial datasets for your exact prep and stimulation artifacts are usually the limiting factor.

Suitability. Use only after you have a strong feature-based baseline; then add a tiny classifier to reduce false positives.

Recommended final detection pipeline for STM32H723ZG

This is an RNS-inspired hybrid optimized for IED-like transient detection with low latency.

1. Input conditioning

- DC removal / high-pass (e.g., 1–2 Hz) to mitigate baseline drift.
- Notch at 50/60 Hz only if needed (beware ringing).

2. IED bandpass (causal IIR biquads)

- Primary: **20–80 Hz** or **15–100 Hz** (starting point for iEEG/LFP spikes; tune empirically). Use 2–4 biquads total.
- Optional parallel band for slow sharp waves: **5–30 Hz** if you see prominent slow components.

3. Feature extraction (per-sample O(1))

- **Line length** over 8–32 ms window (primary trigger feature) ([1.2], [1.3]).
- **Half-wave counting** with amplitude+duration gates (adds morphology constraint) ([1.1]).
- Optional **area/RMS** as a third feature if you need additional robustness ([1.3]).

4. Adaptive thresholding

- Use robust thresholds relative to background: conceptually aligned with RNS short-vs-long background comparison ([2.1]).
- Practical embedded version: hop-domain running median + MAD approximation (or mean+std if you can accept outlier sensitivity). Update baseline only when not in artifact/refractory and when detector is “quiet.”

5. Decision logic (hybrid)

- Trigger when **LL > Thr_LL** AND **HW_count > Thr_HW** for ≥ 1 hop.
- Optionally require envelope/area gate to reject tiny noise bursts.

6. Artifact rejection for stimulation

- Hard blanking: ignore detection for 5–10 ms after each stim pulse/burst.
- Hold baseline adaptation for an additional ~10–50 ms to avoid contaminating adaptive thresholds.

7. Refractory / lockout

- For single-spike stimulation: 50–150 ms lockout is a typical starting range to prevent retriggering on spike aftercomponents or residual artifact.

Embedded-oriented pseudocode

```
// Embedded-friendly pseudocode for causal IED detection on STM32H723ZG
// Pipeline: bandpass -> line length + half-wave hybrid -> robust adaptive threshold
// -> artifact blanking -> refractory/lockout
// Designed for per-sample update, hop-based feature evaluation, ring buffers, and incremental sums.
// Rationale: line length and half-wave style detection are used in implantable real-time systems,
// including RNS-style detectors; short-vs-long window logic is hardware-friendly ([1.2], [1.1], [2.1], [1.3]).

# include
# include

// -----
// Configuration
// -----
# define FS_HZ          1000      // example; 500-2000 Hz practical
# define N_CH           1        // per-channel detector instance

// Bandpass target for IED/spike-rich intracranial/LFP activity.
// Implement as low-order causal IIR biquads or FIR if needed.
// Example practical target: 20-80 Hz or 15-100 Hz, tune per prep.

# define LL_WIN_MS       16       // short feature window for line length
# define HOP_MS          4        // evaluate every 4 ms
# define BASELINE_WIN_MS 10000    // 10 s adaptive baseline
# define REFRACTORY_MS   80       // lockout after detection
# define ARTIFACT_BLANK_MS 8      // post-stim blanking
# define ARTIFACT_HOLD_MS 20      // freeze thresholds after artifact
# define HALF_WAVE_MIN_MS 4       // reject ultra-fast noise/stim edges
# define HALF_WAVE_MAX_MS 30      // typical spike half-wave bound
# define MIN_LL_AND_HW_CONCUR 1  // require both to agree in same hop

# define LL_WIN_SAMPLES  ((FS_HZ * LL_WIN_MS) / 1000)
# define HOP_SAMPLES      ((FS_HZ * HOP_MS) / 1000)
# define BASELINE_LEN     ((BASELINE_WIN_MS) / (HOP_MS))
# define REFRACTORY_SAMPLES ((FS_HZ * REFRACTORY_MS) / 1000)
# define ARTIFACT_BLANK_SAMPLES ((FS_HZ * ARTIFACT_BLANK_MS) / 1000)
# define ARTIFACT_HOLD_SAMPLES ((FS_HZ * ARTIFACT_HOLD_MS) / 1000)
# define HW_MIN_SAMPLES   ((FS_HZ * HALF_WAVE_MIN_MS) / 1000)
# define HW_MAX_SAMPLES   ((FS_HZ * HALF_WAVE_MAX_MS) / 1000)

// Robust threshold multipliers; tune per animal/prep
# define K_LL             6        // threshold = median + K * MAD
# define K_HW             5
# define K_AMP            6        // optional amplitude sanity gate

// Absolute artifact guards in filtered or raw units; tune experimentally
# define RAW_ARTIFACT_ABS  5000    // ADC units or uV-scaled int
# define FILT_SLOPE_ARTIFACT_ABS 3000

// -----
// Simple ring buffer
// -----
typedef struct {
    int32_t *buf;
    uint16_t len;
    uint16_t idx;
} RingI32;
```

```

static inline void ring_push(RingI32 *r, int32_t x) {
    r->buf[r->idx] = x;
    r->idx++;
    if (r->idx >= r->len) r->idx = 0;
}

static inline int32_t ring_get_prev(const RingI32 *r, uint16_t back) {
    int32_t i = (int32_t)r->idx - 1 - (int32_t)back;
    while (i < 0) i += r->len;
    return r->buf[i];
}

// -----
// Running robust baseline
// Exact running median is expensive; practical MCU choices:
//   A) hop-domain histogram median if feature range is bounded
//   B) P2 quantile estimator for median + EWMA abs deviation
//   C) small sorted buffer updated every hop if BASELINE_LEN modest
// Here we show a bounded histogram median + MAD approximation.
// -----
#define FEAT_BINS 256

typedef struct {
    uint16_t hist[FEAT_BINS];
    uint8_t bin_ring[BASELINE_LEN];
    uint16_t ring_idx;
    uint16_t count;
    int32_t f_min;
    int32_t f_max;
    int32_t median;
    int32_t mad; // approximated as median(|x-median|) via EW update or 2nd histogram
    int32_t mad_ew; // cheap fallback if full MAD histogram omitted
} RobustTracker;

static inline uint8_t quantize_feature(int32_t x, int32_t fmin, int32_t fmax) {
    if (x <= fmin) return 0;
    if (x >= fmax) return FEAT_BINS - 1;
    int32_t num = (x - fmin) * (FEAT_BINS - 1);
    int32_t den = (fmax - fmin);
    return (uint8_t)(num / den);
}

static int32_t dequantize_feature(uint8_t b, int32_t fmin, int32_t fmax) {
    return fmin + ((int32_t)b * (fmax - fmin)) / (FEAT_BINS - 1);
}

static void robust_init(RobustTracker *rt, int32_t fmin, int32_t fmax, int32_t mad0) {
    for (int i = 0; i < FEAT_BINS; i++) rt->hist[i] = 0;
    rt->ring_idx = 0;
    rt->count = 0;
    rt->f_min = fmin;
    rt->f_max = fmax;
    rt->median = fmin;
    rt->mad = mad0;
    rt->mad_ew = mad0;
    for (int i = 0; i < BASELINE_LEN; i++) rt->bin_ring[i] = 0;
}

static void robust_push(RobustTracker *rt, int32_t x) {
    uint8_t b = quantize_feature(x, rt->f_min, rt->f_max);

```

```

    if (rt->count < BASELINE_LEN) {
        rt->bin_ring[rt->ring_idx] = b;
        rt->hist[b]++;
        rt->count++;
    } else {
        uint8_t oldb = rt->bin_ring[rt->ring_idx];
        if (rt->hist[olddb] > 0) rt->hist[olddb]--;
        rt->bin_ring[rt->ring_idx] = b;
        rt->hist[b]++;
    }

    rt->ring_idx++;
    if (rt->ring_idx >= BASELINE_LEN) rt->ring_idx = 0;

    // recompute median at hop rate only
    uint32_t acc = 0;
    uint32_t target = rt->count >> 1;
    uint8_t medb = 0;
    for (uint16_t i = 0; i < FEAT_BINS; i++) {
        acc += rt->hist[i];
        if (acc >= target) { medb = (uint8_t)i; break; }
    }
    rt->median = dequantize_feature(medb, rt->f_min, rt->f_max);

    // cheap MAD approximation: EWMA of |x-median|
    // If needed, replace with second histogram over abs deviations.
    int32_t ad = x - rt->median;
    if (ad < 0) ad = -ad;
    rt->mad_ew += (ad - rt->mad_ew) >> 5;    // tau ~ 32 hops
    rt->mad = rt->mad_ew;
    if (rt->mad < 1) rt->mad = 1;
}

static inline int32_t robust_threshold(const RobustTracker *rt, int32_t k) {
    return rt->median + k * rt->mad;
}

// -----
// Bandpass filter state
// Replace with CMSIS-DSP biquad cascade in implementation.
// -----
typedef struct {
    float z1, z2;
    float b0, b1, b2, a1, a2;
} Biquad;

static inline int32_t biquad_step_q(Biquad *s, int32_t x_q) {
    float x = (float)x_q;
    float y = s->b0*x + s->z1;
    s->z1 = s->b1*x - s->a1*y + s->z2;
    s->z2 = s->b2*x - s->a2*y;
    return (int32_t)y;
}

// -----
// Detector state
// -----
typedef struct {
    // Filtering
    Biquad bp1;

```

```

Biquad bp2;

// Sample history
RingI32 x_filt_ring;
int32_t x_filt_storage[LL_WIN_SAMPLES + 8];
int32_t prev_filt;
int32_t prev_diff_abs;

// Incremental line length over LL window
RingI32 diff_abs_ring;
int32_t diff_abs_storage[LL_WIN_SAMPLES + 8];
int64_t ll_sum;

// Optional short amplitude envelope for gating
int32_t env;

// Half-wave extraction
int32_t last_x;
int32_t last_dx;
int32_t extremum_value;
uint32_t extremum_index;
int32_t last_extremum_value;
uint32_t last_extremum_index;
uint16_t hw_count_win;
RingI32 hw_ring;
int32_t hw_storage[HOP_SAMPLES + 8];

// Time / state
uint32_t sample_idx;
uint16_t hop_countdown;
uint32_t refractory_until;
uint32_t artifact_blank_until;
uint32_t artifact_hold_until;

// Robust adaptive baselines in hop domain
RobustTracker ll_base;
RobustTracker hw_base;
RobustTracker env_base;

// Outputs
bool detected;
uint32_t detected_sample;
} IEDDetector;

// -----
// Init
// -----
void ied_init(IEDDetector *d) {
    d->x_filt_ring.buf = d->x_filt_storage;
    d->x_filt_ring.len = LL_WIN_SAMPLES + 8;
    d->x_filt_ring.idx = 0;

    d->diff_abs_ring.buf = d->diff_abs_storage;
    d->diff_abs_ring.len = LL_WIN_SAMPLES + 8;
    d->diff_abs_ring.idx = 0;

    d->hw_ring.buf = d->hw_storage;
    d->hw_ring.len = HOP_SAMPLES + 8;
    d->hw_ring.idx = 0;

    for (uint16_t i = 0; i < d->x_filt_ring.len; i++) d->x_filt_storage[i] = 0;

```

```

    for (uint16_t i = 0; i < d->diff_abs_ring.len; i++) d->diff_abs_storage[i] = 0;
    for (uint16_t i = 0; i < d->hw_ring.len; i++) d->hw_storage[i] = 0;

    d->prev_filt = 0;
    d->prev_diff_abs = 0;
    d->ll_sum = 0;
    d->env = 0;

    d->last_x = 0;
    d->last_dx = 0;
    d->extremum_value = 0;
    d->extremum_index = 0;
    d->last_extremum_value = 0;
    d->last_extremum_index = 0;
    d->hw_count_win = 0;

    d->sample_idx = 0;
    d->hop_countdown = HOP_SAMPLES;
    d->refractory_until = 0;
    d->artifact_blank_until = 0;
    d->artifact_hold_until = 0;

    robust_init(&d->ll_base, 0, 50000, 100);
    robust_init(&d->hw_base, 0, 64, 1);
    robust_init(&d->env_base, 0, 20000, 50);

    d->detected = false;
    d->detected_sample = 0;
}

// -----
// Artifact detection / blanking
// Call this on stimulation onset too.
// -----
static inline void trigger_artifact_blanking(IEDDetector *d) {
    d->artifact_blank_until = d->sample_idx + ARTIFACT_BLANK_SAMPLES;
    d->artifact_hold_until = d->sample_idx + ARTIFACT_HOLD_SAMPLES;
}

static inline bool sample_is_artifact(IEDDetector *d, int32_t raw_x, int32_t filt_x) {
    int32_t slope = filt_x - d->prev_filt;
    int32_t a_raw = (raw_x >= 0) ? raw_x : -raw_x;
    int32_t a_slp = (slope >= 0) ? slope : -slope;
    if (a_raw > RAW_ARTIFACT_ABS || a_slp > FILT_SLOPE_ARTIFACT_ABS) {
        trigger_artifact_blanking(d);
        return true;
    }
    return (d->sample_idx < d->artifact_blank_until);
}

// -----
// Half-wave tracker
// Detect extrema via derivative sign change.
// Count qualifying half-waves inside current hop.
// -----
static inline void update_halfwave(IEDDetector *d, int32_t x) {
    int32_t dx = x - d->last_x;
    int32_t hw_event = 0;

    // sign change => extremum at previous sample
    if ((d->last_dx > 0 && dx <= 0) || (d->last_dx < 0 && dx >= 0)) {

```

```

    int32_t ext_val = d->last_x;
    uint32_t ext_idx = d->sample_idx - 1;

    int32_t amp = ext_val - d->last_extremum_value;
    if (amp < 0) amp = -amp;
    uint32_t dur = ext_idx - d->last_extremum_index;

    if (dur >= HW_MIN_SAMPLES && dur <= HW_MAX_SAMPLES) {
        int32_t amp_thr = robust_threshold(&d->env_base, K_AMP);
        if (amp >= amp_thr) {
            hw_event = 1;
        }
    }

    d->last_extremum_value = ext_val;
    d->last_extremum_index = ext_idx;
}

// Maintain count within last hop by storing per-sample 0/1 flags
int32_t old = ring_get_prev(&d->hw_ring, HOP_SAMPLES - 1);
d->hw_count_win -= (uint16_t)old;
ring_push(&d->hw_ring, hw_event);
d->hw_count_win += (uint16_t)hw_event;

d->last_dx = dx;
d->last_x = x;
}

// -----
// Main per-sample update
// Returns true on detection event.
// -----
bool ied_step(IEDDetector *d, int32_t raw_x, bool stim_now) {
    d->detected = false;

    if (stim_now) {
        trigger_artifact_blanking(d);
    }

    // 1) causal bandpass
    int32_t x = biquad_step_q(&d->bp1, raw_x);
    x = biquad_step_q(&d->bp2, x);

    // 2) sample-level artifact handling
    bool in_artifact = sample_is_artifact(d, raw_x, x);

    // 3) update short amplitude envelope (cheap rectified EWMA)
    int32_t ax = (x >= 0) ? x : -x;
    d->env += (ax - d->env) >> 3;    // ~8-sample smoothing

    // 4) update line length incrementally
    int32_t diff = x - d->prev_filt;
    int32_t diff_abs = (diff >= 0) ? diff : -diff;

    int32_t old_diff_abs = ring_get_prev(&d->diff_abs_ring, LL_WIN_SAMPLES - 1);
    d->ll_sum -= old_diff_abs;
    ring_push(&d->diff_abs_ring, diff_abs);
    d->ll_sum += diff_abs;

    ring_push(&d->x_filt_ring, x);

```

```

// 5) update half-wave logic
update_halfwave(d, x);

// 6) hop-domain decision only every HOP_SAMPLES
d->hop_countdown--;
if (d->hop_countdown == 0) {
    d->hop_countdown = HOP_SAMPLES;

    int32_t ll_feat = (int32_t)(d->ll_sum / LL_WIN_SAMPLES);
    int32_t hw_feat = d->hw_count_win;
    int32_t env_feat = d->env;

    // Freeze baseline updates during artifact hold and refractory if desired
    bool freeze_baseline = (d->sample_idx < d->artifact_hold_until);

    // Robust adaptive thresholds
    int32_t ll_thr = robust_threshold(&d->ll_base, K_LL);
    int32_t hw_thr = robust_threshold(&d->hw_base, K_HW);
    int32_t env_thr = robust_threshold(&d->env_base, K_AMP);

    bool ll_hit = (ll_feat > ll_thr);
    bool hw_hit = (hw_feat > hw_thr);
    bool env_hit = (env_feat > env_thr);

    // Hybrid rule:
    // - require line length increase
    // - require at least one qualifying half-wave in same hop
    // - optional envelope gate to suppress tiny/noisy detections
    bool candidate = ll_hit && hw_hit && env_hit;

    bool locked_out = (d->sample_idx < d->refractory_until);

    if (!in_artifact && !locked_out && candidate) {
        d->detected = true;
        d->detected_sample = d->sample_idx;
        d->refractory_until = d->sample_idx + REFRACTORY_SAMPLES;
    }

    // Update adaptive baselines only using presumed background
    // Exclude artifacts, detections, and refractory intervals to avoid contamination.
    bool safe_for_baseline = (!freeze_baseline) && (!candidate) && (!locked_out) &&
(!in_artifact);
    if (safe_for_baseline) {
        robust_push(&d->ll_base, ll_feat);
        robust_push(&d->hw_base, hw_feat);
        robust_push(&d->env_base, env_feat);
    }
}

d->prev_filt = x;
d->sample_idx++;
return d->detected;
}

// -----
// Notes for embedded C/C++ implementation:
// 1) Prefer CMSIS-DSP biquad cascade for bandpass.
// 2) For fastest deterministic timing, convert filter and features to fixed-point
(Q15/Q31)
// if ADC dynamic range and gains are known; otherwise float is acceptable on Cortex-M7
FPU.

```

```
// 3) Keep baseline trackers in hop domain, not sample domain, to reduce CPU and RAM.
// 4) Start with a single channel and single detector instance; add channel voting later if
//     needed.
// 5) Tune using labeled IEDs: sweep band, LL_WIN_MS, HOP_MS, K_LL, K_HW, refractory, and
//     artifact blanking.
// 6) If stimulation artifact is strong, assert stim_now from the stim scheduler and op-
//     tionally require
//     a post-blank recovery period before re-arming threshold adaptation ([1.2], [2.1]).
```

Code_block: This code block gives embedded-oriented pseudocode for a causal STM32 IED detector that combines bandpass filtering, line length, and half-wave counting with robust adaptive thresholds, artifact blanking, and refractory logic. It is useful as a direct implementation template for C/C++ development on a Cortex-M7 MCU.

Suggested parameter starting points

(These are engineering starting points; final values must be tuned per preparation and electrode site.)

Suggested filter bands

- IED/spike-rich iEEG/LFP: **15–100 Hz** (broad) or **20–80 Hz** (narrower, less EMG) (aligns with RNS operating on band-limited features and half-wave counts) ([1.1], [2.1]).
- If you observe strong slow sharp waves: add a parallel **5–30 Hz** detector and OR/AND with the fast-band detector.

Suggested window sizes

- LL window: **8–32 ms** (spike-like transients); hop 2–5 ms.
- Half-wave duration constraints: **~4–30 ms** (tune to your spikes).
- Baseline adaptation: **5–30 s** (or longer if stable), updated at hop rate.

Suggested adaptive thresholds

- Baseline relative thresholding (RNS pattern): short-term metric compared to long-term background metric ([2.1]).
- Robust variant (recommended): threshold = median + K·MAD of background-only feature values.

Validation and tuning protocol (IED-specific)

Metrics to compute

- **Sensitivity**: fraction of labeled IEDs detected within a tolerance window (e.g., ± 20 ms around the expert-marked peak).
- **False positives per minute**: count detections not matched to any labeled IED.
- **Specificity**: for event detection, report FP/min or FP/hour; standard “specificity” depends on how you define non-event windows.
- **Detection latency**: time between labeled IED peak (or onset) and trigger time.

How to validate

1. Collect iEEG/LFP segments with manual IED labels (and mark stimulation times).
2. Run the detector causally in replay, identical to MCU streaming.
3. Match detections to labels with a bipartite matching (one detection per IED).
4. Report sensitivity, FP/min, and latency distributions; stratify by behavioral state and by “near stim” vs “far from stim.”

How to tune

- Start by fixing the bandpass and LL window; sweep K_LL to reach target FP/min.
- Add half-wave constraints and adjust K_HW / duration bounds to recover sensitivity at the same FP/min.
- Finally tune artifact blanking and refractory to minimize stim-locked false detections.

Practical STM32H723ZG implementation plan

Numeric format

- Cortex-M7 includes an FPU; **float32** biquad filtering via CMSIS-DSP is often simplest and fast enough.
- For deterministic timing and lower power, consider **Q31 fixed-point** for filters and features once gains and ADC scaling are stable. Features like line length and half-wave counts are naturally integer-friendly (abs, sums).

Memory plan (per channel)

- Ring buffers for LL window (|diff) and for hop window half-wave flags.
- Baseline tracker in hop domain (e.g., bounded histogram or small sorted buffer).

Real-time scheduling

- Per-sample ISR/DMA callback: filter + O1 feature updates.
- Hop task every 2–5 ms: evaluate thresholds/logic, update baseline if safe.
- Stimulation task: asserts stim_now flag; triggers artifact blanking immediately.

Clear distinctions: IED vs seizure vs offline

- **IED-specific intracranial spike detection evidence:** intracranial spike detection/quantification workflows and comparisons exist (Dümpelmann & Elger 1999; Hufnagel 2000), but agreement even between human reviewers can be modest, highlighting label ambiguity and the need for patient/prep-specific tuning ([5.2], [6.2]).
- **Seizure-validated lightweight features:** time-domain features (energy/RMS/LL/coastline) are emphasized for implantable feasibility ([3.1]), and TKEO threshold detectors show variable sensitivity/specificity across seizure models ([7.2]).
- **Offline / non-embedded:** DTW template matching is highly effective for rapid IED annotation (useful for your labeling pipeline) but is not a typical MCU closed-loop detector ([9.3]). Wavelet spike detectors have real-time implementations but historically used dedicated DSP cards and often lack reported embedded resource metrics ([10.1], [11.1]).

Bottom line recommendation

For a microcontroller-based, low-latency closed-loop IED trigger, the best tradeoff is a **hybrid line-length + bandpass half-wave detector with robust adaptive thresholds and explicit stim-artifact blanking**, because these primitives are (i) causal and computationally efficient, and (ii) strongly supported by implantable closed-loop practice (RNS) rather than only offline spike-detection papers ([1.2], [1.1], [2.1]).

References

[1.1] The RNS System: responsive cortical stimulation for the treatment of refractory partial epilepsy. Felice T Sun, Martha J Morrell. Expert Review of Medical Devices (2014). <https://doi.org/10.1586/17434440.2014.947274> Context: "Excerpt provides NeuroPace RNS technical details: sensed ECoG is digitized at 250 Hz and up to four bipolar channels can be sensed simultaneously. Detection is implemented to run in real time within implant power/processing limits. The system offers three configurable detection tools (line length, area and bandpass) and allows up to two independent detectors per sensing channel. Line length is defined as the sum of sample-to-sample differences within a window divided by number of samples; detection compares a short-term sliding-window measurement to a long-term background window and triggers when the short-term exceeds an absolute or relative threshold. The bandpass tool segments the signal into local minima/maxima half-waves; half-waves exceeding physician-programmed amplitude and duration are counted and must surpass a count threshold within a window to trigger detection. The bandpass tool functions as a frequency filter for bands (theta/alpha/beta/gamma). The excerpt does not provide details on Dümpelmann/Elger/Hufnagel interictal spike algorithms or on specific DSP/microprocessor implementations (e.g., Clarençon 1996, Panych 2010)."

[1.2] The RNS System: responsive cortical stimulation for the treatment of refractory partial epilepsy. Felice T Sun, Martha J Morrell. Expert Review of Medical Devices (2014). <https://doi.org/10.1586/17434440.2014.947274> Context: "The excerpt describes NeuroPace RNS detection tools and stimulation settings. The system implements line length, bandpass and area detectors. Line length compares average line length in a short-term sliding window to a long-term sliding window and triggers when the short-term measure exceeds an absolute or relative threshold; negative thresholds can detect decreases (e.g., electrodecremental events). Because line length is sensitive to both amplitude and frequency it is often used as an initial broad detector. The area detector measures overall signal energy regardless of frequency. The bandpass detector provides more specific detection than line length or area and is used once ECoG patterns/frequency bands (e.g., theta/alpha/beta/gamma) of interest are identified. Outputs from multiple detectors and channels can be combined, and each detector output can have a persistence duration to detect sequences (example: spike-wave elements detected by line length followed by bandpass-detected high-frequency activity). The neurostimulator parameters are given (biphasic pulses; pulses 40–1000 ms; frequencies 1–333 Hz; currents 0.5–12 mA; burst durations 10–5000 ms) and typical initial therapy settings (200 Hz, 160 ms pulse width, 100 ms burst). The excerpt does not provide details on half-wave detectors, the Dümpelmann/Elger/Hufnagel intracranial interictal spike algorithms, performance/accuracy metrics, nor on dedicated real-time DSP/microprocessor implementations such as Clarençon 1996 or Panych 2010."

[1.3] The RNS System: responsive cortical stimulation for the treatment of refractory partial epilepsy. Felice T Sun, Martha J Morrell. Expert Review of Medical Devices (2014). <https://doi.org/10.1586/17434440.2014.947274> Context: "The excerpt documents lightweight, real-time detection features implemented in the RNS neurostimulator suitable for implantable hardware. Signals are digitized at 250 Hz. Detection algorithms are described as "computationally efficient" and optimized for the power/processing constraints of implantable devices. Three configurable detector types are provided: line length, area ("defined as the average area under the curve within a window"), and a bandpass/half-wave detector. Line length is computed as the sum of sample-to-sample differences normalized by sample count and is compared between a short-term sliding window and a long-term background window; detection is triggered when the short-term value exceeds an absolute or relative threshold versus the long-term value. The bandpass detector segments the signal into half-waves (local minima/maxima), counts half-waves that exceed programmed amplitude/duration, and triggers when counts in a window exceed a threshold. Up to two independent detectors can run per sensing channel. The excerpt does not report sensitivity/specificity/false-positive rates nor mention TKEO/NEO/SNEO, envelope/rectification, derivative/slope, or bandpower-ratio metrics; it also does not provide precise window lengths or latency numbers, but the descriptions imply causal sliding-window computations designed for low-power embedded implementation."

[2.1] Closed-loop Neurostimulation: The Clinical Experience. Felice T. Sun, Martha J. Morrell. Neurotherapeutics (2014). <https://doi.org/10.1007/s13311-014-0280-3> Context: "The excerpt describes the NeuroPace RNS detection tools: three lightweight, configurable detectors—bandpass, line-length, and area—designed for implantable hardware with limited power/processing. The bandpass detector (Gotman-like) detects spikes and rhythmic activity in specific frequency bands using "half-waves" (segments partitioned at local minima and maxima). The line-length detector measures the average of absolute sample-to-sample amplitude differences within a window and identifies changes in amplitude and frequency. The area detector is an energy/power measure defined as the average absolute area-under-the-curve and flags overall signal energy changes regardless of frequency. Detection uses a short-term vs long-term sliding-window comparison (short-term: 128 ms–4 s; long-term: 4 s–16 min); detection occurs when the short-term metric crosses an absolute or relative threshold derived from the long-term background. Algorithms are computationally efficient, configurable for sensitivity/specificity/latency, and can

detect events from fractions of a second up to several seconds. The excerpt does not provide performance metrics (accuracy, sensitivity/specificity values) for these detectors, nor details on Dümpelmann/Elger/Hufnagel intracranial spike algorithms or any real-time DSP/microprocessor implementations mentioned (e.g., Clarençon 1996, Panych 2010)."

[3.1] A hardware-algorithm co-design approach to optimize seizure detection algorithms for implantable applications. Shriram Raghunathan, Sumeet K. Gupta, Himanshu S. Markandeya, Kaushik Roy, Pedro P. Irazoqui. *Journal of Neuroscience Methods* (2010). <https://doi.org/10.1016/j.jneumeth.2010.08.008> Context: "The excerpt documents a set of lightweight, time-domain features used for seizure detection that are computationally simple and suitable for embedded/implantable hardware: energy ($E[i] = x^2(i)$ and windowed averages $Eavg[k]$), RMS amplitude ($A[k]$ computed from squared samples), coastline/line-length (sum of absolute differences between consecutive samples, $CL(k)$), Hjorth variance parameter, rectified variance (sum of absolute deviations from mean), and a non-linear autocorrelation computed from local extrema. These features are computed with sliding rectangular windows (window index k , window size N), using basic operations: squaring, summation, absolute value, max/min, and simple averaging — all causal and amenable to efficient hardware implementation. The paper notes multi-scale windowing (example: 18,000-sample windows composed of 180-sample subwindows for the autocorrelation procedure) and cites that an implantable device (Neuropace) uses line length in its detection algorithm, reinforcing suitability for implants. The excerpt does not report sensitivity, specificity, or false-positive rates for these features, nor does it discuss bandpower ratios or energy-operator variants (TKEO/NEO/SNEO); those items are not present in the provided text."

[3.2] A hardware-algorithm co-design approach to optimize seizure detection algorithms for implantable applications. Shriram Raghunathan, Sumeet K. Gupta, Himanshu S. Markandeya, Kaushik Roy, Pedro P. Irazoqui. *Journal of Neuroscience Methods* (2010). <https://doi.org/10.1016/j.jneumeth.2010.08.008> Context: "This paper evaluates seizure-detection features from a joint hardware–algorithm viewpoint to identify lightweight real-time measures suitable for implantable devices. It explicitly compares simple features (example list includes "area, line length and half-wave") and assigns each a detection-efficacy score and a hardware cost computed from power consumption and silicon area, then plots them in a two-dimensional detection algorithm design space (DADS). The authors note that implementations on standard microcontrollers are feasible for low-complexity features, while FFT/DWT and other memory/compute-intensive methods require DSPs and have high power/memory cost. Memory-intensive operations and wireless transmission negatively impact battery life; integrating detection at the recording electrodes reduces transmission needs. Thermal limits ("devices that dissipate over 6 mW/sq cm can generate enough heat to damage surrounding tissue") and battery constraints are emphasized. The methods were tested on seizure data from five kainate-treated rats. The paper also shows simple block/casual computations (e.g., an 'activity' parameter defined as the variance of signal amplitude), supporting sliding-window blockwise processing suitable for embedded implementations. The excerpt does not provide explicit sensitivity/specificity or false-positive numeric results."

[4.1] Closed-Loop Neural Prostheses With On-Chip Intelligence: A Review and a Low-Latency Machine Learning Model for Brain State Detection. Bingzhao Zhu, Uisub Shin, Mahsa Shoaran. *IEEE Transactions on Biomedical Circuits and Systems* (2021). <https://doi.org/10.1109/tbcas.2021.3112756> Context: "The excerpt provides a compact hardware-oriented listing of lightweight features used for epileptiform detection and their on-chip cost estimates. Table III enumerates bandpower features across canonical EEG bands (delta through ripple) with a per-feature power cost of 250.6 (nW) and latencies ranging from 0.25 s (beta and gamma bands) up to 1 s (delta). Line-Length (LLN) is explicitly listed with much lower power (7.4 nW) and 0.25 s latency; the excerpt also shows the line-length difference-based computation form ($d \sum |x[n] - x[n-1]|$, window size), indicating a causal, sample-difference implementation suitable for low-compute embedded hardware. Table II notes feature types including HFO and BPF and shows sensing sampling rates up to 5 kS/s per channel, implying feasibility for higher-frequency feature extraction (e.g., ripples/HFO). The excerpt does not report performance metrics (sensitivity/specificity/false-positive rates) nor explicit mentions of envelope/rectification, RMS/energy/area, derivative/slope beyond the line-length difference, or TKEO/NEO/SNEO. Also absent are detailed descriptions of causal sliding-window sizes beyond the line-length window hint and concrete implementation kernels, though the listed power/latency values support selection of computationally cheap, causal features for implantable/embedded detectors."

[5.1] Visual and Automatic Investigation of Epileptiform Spikes in Intracranial EEG Recordings. Matthias Dümpelmann, Christian E. Elger. *Epilepsia* (1999). <https://doi.org/10.1111/j.1528-1157.1999.tb00704.x> Context: "The excerpt describes intracranial interictal spike detection methods used and compared in Dümpelmann et al. (1999). For the rule-based Gotman detector (software V.6.0) an adjustable amplitude threshold was tested "from 2 to 20 in steps of 2"; additional criteria include peak sharpness, durations of the inclining and declining parts, and "Phase reversal in neighboring channels increases the possibility of detecting a spike." EEG segments with probable spikes were stored and "quantified spike parameters are stored, including the channel and the time of the spike occurrence." The two-stage system (2SSD) uses a linear autoregressive predictor: model order selection indicated "a suitable range between five and 15," and the study used "a model with an order of 10" to reduce computation. The AR coefficients were estimated via Yule–Walker and "the model coefficients were updated every

4 s." Detection relies on the prediction error: "Spikes in the EEG segment exhibit an increased prediction error," and "the prediction error was squared and averaged over some data points and compared with the squared and averaged prediction error of a greater number of surrounding data points" to form a detection function $T(t)$ which is thresholded. A wavelet-based detector used a "Coiflet-6" wavelet at multiple widths. The excerpt does not report numeric performance metrics (sensitivity, specificity, false positive rates) or real-time microprocessor/DSP implementation details."

[5.2] Visual and Automatic Investigation of Epileptiform Spikes in Intracranial EEG Recordings. Matthias Dümpelmann, Christian E. Elger. *Epilepsia* (1999). <https://doi.org/10.1111/j.1528-1157.1999.tb00704.x> Context: "This excerpt (Dümpelmann & Elger 1999) reports intracranial interictal-spike analysis comparing two human reviewers and three automatic detectors (Gotman rule-based, a two-stage linear-predictor + rule-based system, and a wavelet-coefficient detector). Data: intracranial subdural and intrahippocampal depth contacts from seven patients; hardware/filtering: bandpass 0.53–85 Hz (12 dB/oct), A/D 12-bit, sampling 173 Hz. Visual spike criteria: adopted from scalp EEG—"an isolated triangular wave...with a sharp peak and steep slopes"; duration criterion of 40 ms could be exceeded so sharp waves included; both polarities marked in depth recordings. Performance metrics reported: interrater agreement between two human reviewers = 40%; automatic systems' agreement with individual human reviewers: Gotman 24%, wavelet detector 26%, two-stage 32%. The two-stage method detects increased prediction error of a linear predictor during spikes. The paper emphasizes that although absolute agreement rates were low, anatomical localization of spike-generating regions was consistent across visual and automatic methods. The excerpt does not provide NeuroPace RNS-specific details (line length, half-wave, short/long windows, thresholds) nor details of real-time microprocessor/DSP implementations (Clarençon 1996 or Panych 2010)."

[6.1] Clinical Relevance of Quantified Intracranial Interictal Spike Activity in Presurgical Evaluation of Epilepsy. Andreas Hufnagel, Matthias Dümpelmann, Josef Zentner, O. Schijns, C. E. Elger. *Epilepsia* (2000). <https://doi.org/10.1111/j.1528-1157.2000.tb00191.x> Context: "The provided Hufnagel et al. (2000) excerpt describes the intracranial interictal spike detection and clustering workflow used (Dümpelmann/Elger algorithm referenced). ECoG was digitized at sampling intervals of 5.4 or 6.0 ms. Epochs averaged 22 min (7–60 min) and recordings were performed while patients were awake. Spike clustering: clusters defined as discharges in a circumscribed area across neighboring channels. Detection workflow: a channel with many spikes was selected, algorithm-detected spikes were displayed and visually verified, and spike peaks were used as trigger points. Neighboring minima were computed within ± 100 ms around the peak; a 200 ms window was also used for averaging steps (shortened when spikes occurred > 5 Hz to avoid coincident events). Spike amplitude was computed by averaging inclining and declining slopes; events were accepted for averaging if amplitude exceeded 30% of the highest amplitude of the event and minima were detectable within the 200-ms frame. The text alludes to slope/second-derivative criteria (fragment: "second derivative: $> -0.3/\text{ms}^2$], a ratio of < 0.9 for incl-in") and notes that all detected spikes were visually verified and clustered. The excerpt does not provide numeric performance metrics (sensitivity/specificity) or details on NeuroPace RNS feature implementations (line length, half-wave, bandpass, window lengths, thresholds) nor on real-time microprocessor/DSP implementations; it refers readers to Dümpelmann and Elger for algorithm details and performance."

[6.2] Clinical Relevance of Quantified Intracranial Interictal Spike Activity in Presurgical Evaluation of Epilepsy. Andreas Hufnagel, Matthias Dümpelmann, Josef Zentner, O. Schijns, C. E. Elger. *Epilepsia* (2000). <https://doi.org/10.1111/j.1528-1157.2000.tb00191.x> Context: "Hufnagel et al. (2000) report a study of 32 patients with intracranial electrocorticographic recordings using a custom computer program for automatic detection of interictal spikes and averaging procedures. The program produced measures at each recording site: earliest spike of each spike cluster, maximal averaged spike amplitude, highest spike frequency, and shortest averaged spike duration. Performance versus the seizure origin (SO) was reported: within a zone "5 2 cm" from SO the averaged earliest spike localized to SO in 27/32 patients (84%), highest averaged amplitude in 24/32 (75%), shortest averaged spike duration in 22/32 (69%), and maximal spike frequency in 17/32 (53%). The authors conclude that quantification of interictal spike activity aids localization of the seizure generator and that removing all areas showing spike clusters is unnecessary for seizure control. The excerpt gives cohort and electrode implantation context (32 patients, various subdural/depth arrays) and electrode contact spacing (1 cm on SDEs and SGEs, 0.5 cm on DEs). The excerpt does not provide algorithmic implementation details (e.g., slope/duration thresholds, time-window lengths, detection thresholds) nor real-time/DSP implementation descriptions."

[7.1] Detection of spontaneous seizures in EEGs in multiple experimental mouse models of epilepsy. Lan Wei, Halima Boutouil, Rogério R Gerbatin, Omar Mamad, Mona Heiland, Cristina R Reschke, Federico Del Gallo, Paolo F Fabene, David C Henshall, Madeleine Lowery, Gareth Morris, Catherine Mooney. *Journal of Neural Engineering* (2021). <https://doi.org/10.1088/1741-2552/ac2ca0> Context: "Relevant lightweight, mostly time-domain features and simple frequency metrics used for real-time/embedded seizure detection in this paper include the Teager-Kaiser Energy Operator (TKEO) (mean TKEO per epoch), band absolute and relative powers (delta, theta, alpha, beta, gamma; total power 0–500 Hz), envelope (mean of the envelope estimated using the Hilbert transform), simple statistics (mean, variance, kurtosis, skewness), mobility and fractal dimension, and DWT-derived bands (7-level db4 decomposition: fast ripple to delta). Data were processed

in short epochs (5 s epochs with 2.5 s overlap). A combined global TKEO-DWT thresholding rule was used: an epoch is a potential seizure if its mean TKEO exceeds the global TKEO threshold and its relative beta power < 0.021 ; events longer than 10 s are labeled seizures. The paper emphasizes TKEO's simplicity and ease of implementation for detecting signal changes. The TKEO is given explicitly ($\text{TKEO}[n] = x[n]^2 - x[n-1]x[n+1]$), which indicates a sample-based computation (note presence of $x[n+1]$ in the formula). The excerpt does not report sensitivity/specificity/false-positive rates or explicit causal sliding-window implementation details or embedded-hardware benchmarks."

[7.2] Detection of spontaneous seizures in EEGs in multiple experimental mouse models of epilepsy. Lan Wei, Halima Boutouil, Rogério R Gerbatin, Omar Mamad, Mona Heiland, Cristina R Reschke, Federico Del Gallo, Paolo F Fabene, David C Henshall, Madeleine Lowery, Gareth Morris, Catherine Mooney. *Journal of Neural Engineering* (2021). <https://doi.org/10.1088/1741-2552/ac2ca0> Context: "This excerpt reports evaluation of a machine-learning detector (Epi-AI) against two threshold-based methods: a local Teager-Kaiser energy operator (TKEO) and a TKEO combined with discrete wavelet transform (TKEO-DWT). Dataset: 26 mice, 6451 h EEG across multiple mouse epilepsy models (Models I–IV). Performance: Epi-AI achieved 91.4%–98.8% sensitivity and 93.1%–98.8% specificity in Models I–III, and 76.3% sensitivity with 98.1% specificity on an unseen Model IV. The TKEO approach showed 66.9%–91.3% sensitivity and 60.8%–97.5% specificity in Models I–III. The TKEO-DWT approach yielded marginally better results than TKEO: 73.2%–80.1% sensitivity and 75.8%–98.1% specificity. The excerpt thus provides quantitative evidence for TKEO-based features (and a TKEO+DWT combo) as lightweight threshold methods with variable sensitivity/specificity across models. The provided pages do not include explicit information about other lightweight features you requested (line length, area/energy/RMS, envelope/rectification, derivative/slope, bandpower ratios, NEO/SNEO), nor do they give causal sliding-window sizes, false-positive rates per hour, computational cost, or implementation details for embedded/implantable hardware."

[8.1] Closed-Loop Neurostimulators: A Survey and A Seizure-Predicting Design Example for Intractable Epilepsy Treatment. Hossein Kassiri, Sana Tonekaboni, M. Tariqus Salam, Nima Soltani, Karim Abdelhalim, Jose Luis Perez Velazquez, Roman Genov. *IEEE Transactions on Biomedical Circuits and Systems* (2017). <https://doi.org/10.1109/tbcas.2017.2694638> Context: "The excerpt surveys seizure-detection/prediction methods relevant to closed-loop devices, emphasizing time-domain and frequency-domain approaches that are amenable to embedded implementation. Time-based methods include instantaneous phase and phase-synchronization metrics (mean phase coherence, bivariate synchrony) and spiking-rate monitoring; frequency-based approaches use spectral power and wavelet energy/entropy. Reported performances include 72.7% sensitivity with 2.64 false positives/day (50 min horizon), 67% sensitivity with ~ 1 false prediction/day (20 min horizon), a highest reported 88% sensitivity with 0.64–4.69 false positives/hour, and an SVM on spectral features achieving 98.3% sensitivity with ~ 7 false positives/day. The authors conclude that patient-specific, small sets of discriminative features plus simple thresholding can match complex AI methods, making low-cost embedded implementations feasible. They note the need to integrate algorithms into chronically-implantable hardware that can record and process continuously (>1 month). The excerpt does NOT explicitly mention lightweight spike/IED features commonly used in implantables (e.g., line length, TKEO, envelope, slope, amplitude/energy) nor specific implementation details like the RNS device's line-length implementation or precise latencies."

[8.2] Closed-Loop Neurostimulators: A Survey and A Seizure-Predicting Design Example for Intractable Epilepsy Treatment. Hossein Kassiri, Sana Tonekaboni, M. Tariqus Salam, Nima Soltani, Karim Abdelhalim, Jose Luis Perez Velazquez, Roman Genov. *IEEE Transactions on Biomedical Circuits and Systems* (2017). <https://doi.org/10.1109/tbcas.2017.2694638> Context: "The excerpt reviews seizure-detection approaches emphasizing lightweight time- and frequency-domain features suitable for real-time systems. Time-domain methods include instantaneous phase and phase-synchronization measures (mean phase coherence, bivariate synchrony) with reported sensitivities up to 88% (false positive rate 0.64–4.69 per hour) and other studies reporting 72.7% sensitivity with 2.64 false positives/day (50 min horizon) or 67% with ~ 1 false prediction/day (20 min horizon). Frequency-based methods use spectral power across multiple bands ("eight frequency bands") and wavelet energy/entropy; one report achieved 98.3% sensitivity with ~ 7 false positives/day after preprocessing and feature selection. The authors highlight that averaging window length, data smoothing, and prediction horizon affect performance, implying causal/sliding-window choices matter. They also note trade-offs: adding many features increases computational cost without guaranteed gains, and patient-specific feature selection plus simple thresholding can yield high accuracy with low cost. Finally, they stress that algorithms must be implementable in chronically implantable devices for long-term (>1 month) recording/processing, favoring computationally light, causal operations and few features. The excerpt does not explicitly mention specific lightweight metrics such as line length, area/RMS, envelope/rectification, derivative/slope, bandpower ratios, or energy operators (TKEO/NEO/SNEO)."

[9.1] Rapid annotation of interictal epileptiform discharges via template matching under Dynamic Time Warping. J. Jing, J. Dauwels, T. Rakthanmanon, E. Keogh, S.S. Cash, M.B. Westover. *Journal of Neuroscience Methods* (2016). <https://doi.org/10.1016/j.jneumeth.2016.02.025> Context: "Jing et al. (2016) describe NeuroBrowser, a semi-automated EEG review tool that uses template matching with Dynamic Time Warping (DTW) to accelerate expert annotation of interictal

epileptiform discharges (IEDs). The system exploits within-patient spike morphological similarity to extract candidate waveforms in groups (typically 10–100 spikes) rather than one-by-one. DTW was chosen over Euclidean Distance because ED is sensitive to small waveform variations. To make DTW practical, the authors adapted the UCR "Trillion" algorithm (custom-modified) for ultra-fast DTW-based similarity search, claiming drastic annotation speed improvement. Experimentally, NeuroBrowser reduced expert annotation time by about 70% compared to unassisted annotation; it was used to build a 19,000+ spike database from 100 patient 30-minute recordings, cross-annotated by multiple neurologists. Practical computational measures include down-sampling EEG to 128 Hz to lower complexity, and standard preprocessing (0.1 Hz high-pass and 60 Hz notch filters). The paper emphasizes speed gains for rapid annotation via fast DTW/template matching but does not report low-level latency numbers (e.g., ms per match) or real-time throughput metrics. No information on Clarençon 1996 or wavelet-DSP real-time spike detectors appears in this excerpt."

[9.2] Rapid annotation of interictal epileptiform discharges via template matching under Dynamic Time Warping. J. Jing, J. Dauwels, T. Rakthanmanon, E. Keogh, S.S. Cash, M.B. Westover. *Journal of Neuroscience Methods* (2016). <https://doi.org/10.1016/j.jneumeth.2016.02.025> Context: "The excerpt describes Jing et al. (2016) implementing DTW-based template matching for interictal epileptiform discharge (IED) annotation in a semi-automated tool (NeuroBrowser). Users manually supply templates and accept/reject candidate matches. The authors replaced Euclidean distance with Dynamic Time Warping (DTW) to allow non-linear time-axis warping and better align spike morphology. They used a custom-modified implementation of the ultra-fast "Trillion" algorithm available in the UCR suite. To make DTW feasible on large EEG datasets, they rely on the UCR suite's speedups (e.g., early abandoning z-normalization) and constrain warping with a Sakoe-Chiba band (width R, typically 10% of signal length). The paper emphasizes that these optimizations enable DTW to be applied to massive datasets. The excerpt, however, does not provide quantitative latency or throughput numbers, nor does it include details on Clarençon 1996 or any real-time wavelet DSP implementation performance metrics."

[9.3] Rapid annotation of interictal epileptiform discharges via template matching under Dynamic Time Warping. J. Jing, J. Dauwels, T. Rakthanmanon, E. Keogh, S.S. Cash, M.B. Westover. *Journal of Neuroscience Methods* (2016). <https://doi.org/10.1016/j.jneumeth.2016.02.025> Context: "The excerpt describes NeuroBrowser, a system for rapid annotation of interictal epileptiform discharges that uses "ultrafast template matching under Dynamic Time Warping" to substantially accelerate expert annotation. The authors report time savings of 20–90% (average ≈70%) for experts using the tool, and state they have extracted over 19,000 IEDs from 100 patient EEG recordings, claiming this as the largest annotated IED database to date. The paper emphasizes the challenge of automated spike detection—large within- and between-patient waveform variability and similarity to background waves and artifacts—and the bottleneck caused by a lack of sufficiently large expert-annotated datasets, since brute-force manual annotation is prohibitively time-consuming. The authors note the algorithm is currently tailored to scalp EEG IED annotation but that the concepts can be generalized to other waveforms. The excerpt does not provide low-level computational details, latency numbers, real-time/DSP implementation specifics, or algorithmic complexity; it reports only overall annotation time reductions and dataset scale."

[10.1] Real-time spike detection in EEG signals using the wavelet transform and a dedicated digital signal processor card. Didier Clarençon, Marc Renaudin, Patrick Gourmelon, Arnaud Kerckhoeve, Richard Catérini, Emmanuel Boivin, Patrick Ellis, Benjamin Hille, Marc Fatôme. *Journal of Neuroscience Methods* (1996). [https://doi.org/10.1016/s0165-0270\(96\)00073-8](https://doi.org/10.1016/s0165-0270(96)00073-8) Context: "The excerpt describes Clarençon et al. (1996), which implements a complete real-time EEG spike-detection system based on the wavelet transform. The authors choose the Morlet wavelet because its shape closely resembles EEG spikes, facilitating faster and easier epileptiform spike characterization. To achieve real-time, multi-channel processing they designed a dedicated DSP card based on the Motorola 96002 processor to support real-time acquisition and processing of pre-filtered channels. The method and hardware were validated on a rat picrotoxin seizure model; epileptiform spikes became detectable in recordings within about 10 minutes after drug injection. They also introduce a "globalization" representation intended to give a global view and improved detection of paroxysmal events. The paper motivates wavelet use by noting Fourier methods' limitations for transient components. The excerpt, however, does not provide low-level computational details, algorithmic complexity, throughput, measured processing latency, or quantitative performance benchmarks (e.g., detection latency, false-positive/false-negative rates) nor comparisons to DTW/template-matching methods like Jing 2016."

[10.2] Real-time spike detection in EEG signals using the wavelet transform and a dedicated digital signal processor card. Didier Clarençon, Marc Renaudin, Patrick Gourmelon, Arnaud Kerckhoeve, Richard Catérini, Emmanuel Boivin, Patrick Ellis, Benjamin Hille, Marc Fatôme. *Journal of Neuroscience Methods* (1996). [https://doi.org/10.1016/s0165-0270\(96\)00073-8](https://doi.org/10.1016/s0165-0270(96)00073-8) Context: "The excerpt describes Clarençon et al. (1996) implementing a real-time EEG spike-detection system based on the wavelet transform. The authors chose the Morlet wavelet because its shape matches EEG spikes and because wavelet multiscale/time-frequency localization suits transient events better than FFT-based analysis. They use a dyadic/wavelet analysis framework and emphasize that the transform is used purely for analysis and detection ("it is not necessary to reconstruct the original signal in any way"), which relaxes reconstruction constraints and reduces processing requirements."

Computational considerations: the algorithm and wavelet scaling/bandwidth can be tuned "keeping the computation time compatible with the real-time constraint." For hardware, they used a Macintosh for control/storage and developed a dedicated modular processing board based on a Motorola 96002 floating-point DSP because commercial boards were inadequate. The excerpt does not provide quantitative latency, throughput, or accuracy numbers, nor any explicit performance benchmarks. The excerpt also contains no information on Jing 2016 or DTW/template-matching IED annotation."

[11.1] Automated Detection of Interictal Spikes in EEG: A literature review. FD De Moraes, DA Callegari. 2014. <https://www.pucrs.br/facim-prov/wp-content/uploads/sites/19/2016/03/tr081.pdf> Context: "The review highlights wavelet transforms as an effective feature-extraction approach for nonstationary EEG signals and notes the trade-offs between methods: CWT gives finer time-frequency resolution but increases computational cost, DWT is less costly, and WPD can be a compromise between resolution and computational demand. Combining feature/decomposition strategies is common. The authors emphasize that many studies do not report essential computational metrics—"memory consumed, processing time"—nor the "algorithm feasibility for real time processing," which hinders assessing latency or suitability for embedded/real-time systems. They state that such computational information is essential for field applications and that developing robust automatic or semi-automatic real-time spike detection remains an open challenge. For supervised classifiers and template-based approaches the paper stresses the need for large, diverse datasets and highly relevant discriminative features to ensure good performance. Overall, while wavelet methods are recommended, concrete latency/performance results and real-time implementation details are generally missing from the reviewed literature."